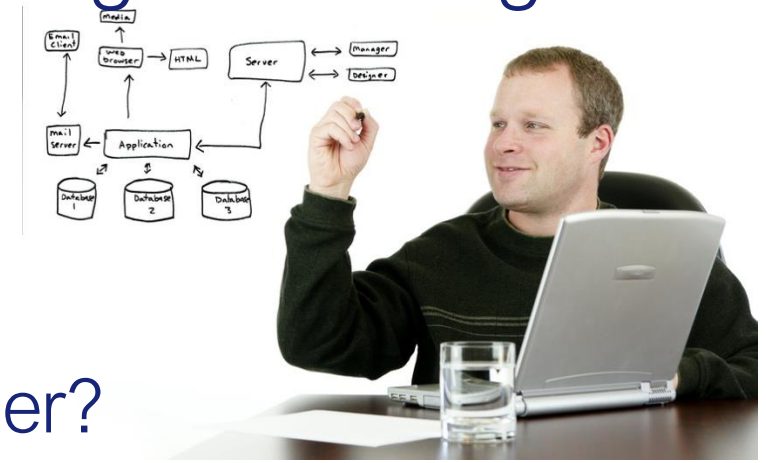
Pair Programming

ESaaS §2.2

Are 2 minds better than 1?

- Stereotype: lone wolf working all night drinking Red Bull
- Is there a more social way to program?
  - What would be benefits of multiple people programming together?

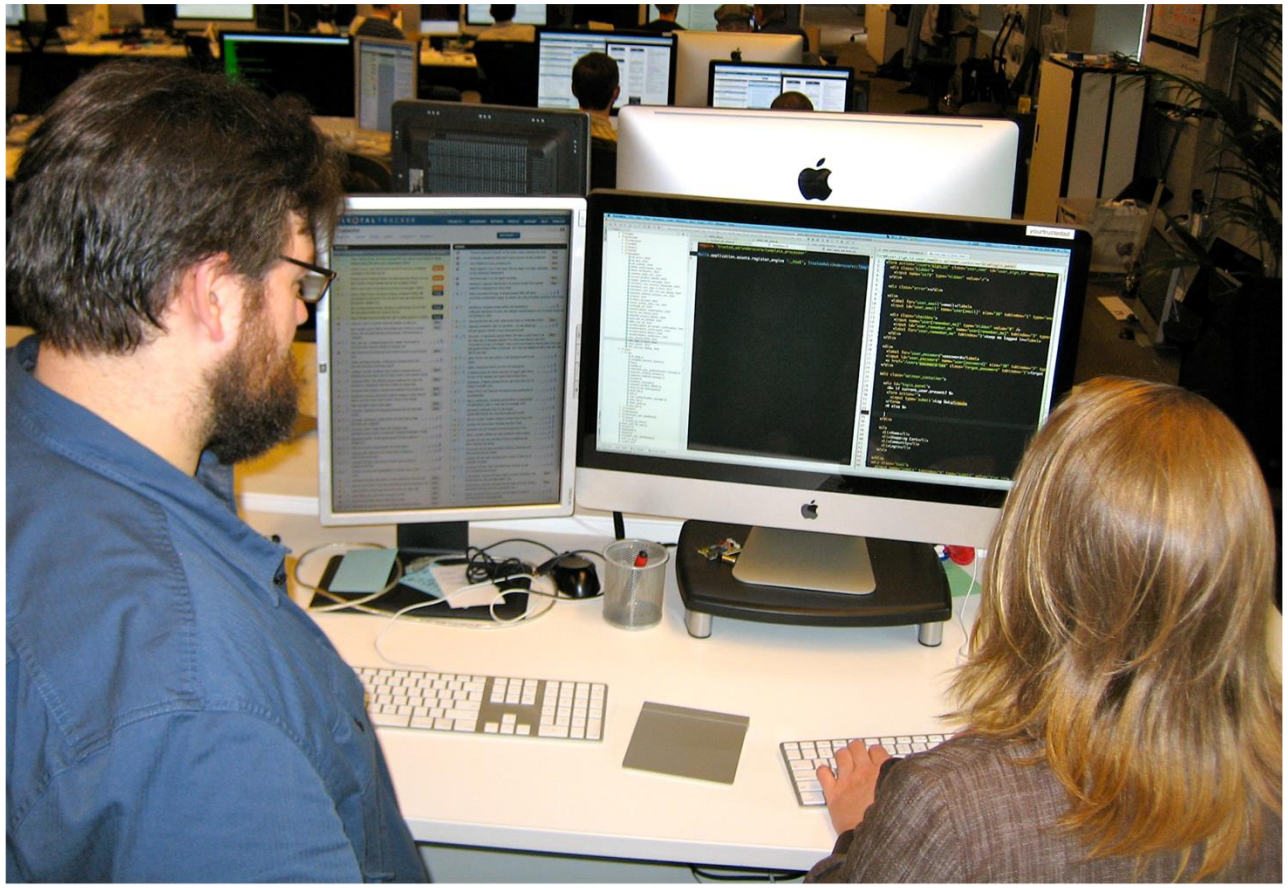


- How would you prevent one person from doing all the work while the other gets coffee and checks (anti)social media?

Pair programming: why do it?

- Goal: improve software quality, reduce time to completion by having 2 people develop the same code
  - Some people (and companies) love it
  - *Try it* to see if you do
  - Some try it and never go back, some find it doesn't help

Pair Programming at Pivotal



Driver and Observer switch roles every 10-15 minutes

Does Pair Programming help?

- Simple tasks ➔ Quicker
- Complex tasks ➔ Yields higher quality code
  - Anecdotally, sometimes more readable too
- More effort than solo programmers
- Keeps both programmers focused
- Transfers knowledge between the pair
  - programming idioms, tool tricks, company processes, latest technologies, …
  - Some teams purposely swap partners per task so eventually everyone pairs with everyone else ("promiscuous pairing")

Do's and Don'ts of Pairing

- Don't be distracted when you are observer
- Do consider pairing with someone of different experience level—you'll both learn!
  - Explaining is a great way to better understand
- Do swap frequently—roles exercise different skills
  - Observer gains skill of explaining their thought process to Driver

Tip #6 from previous students

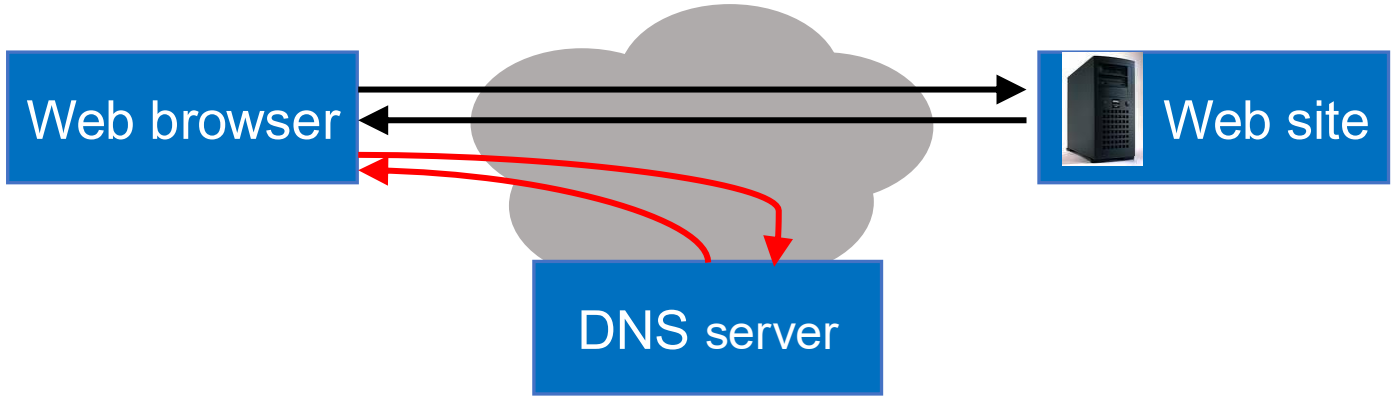
- “Helped avoid silly mistakes that could take a long time to debug”
- “Changing partners frequently made team more cohesive”

The Web's Client-Server Architecture

ESaaS §3.1

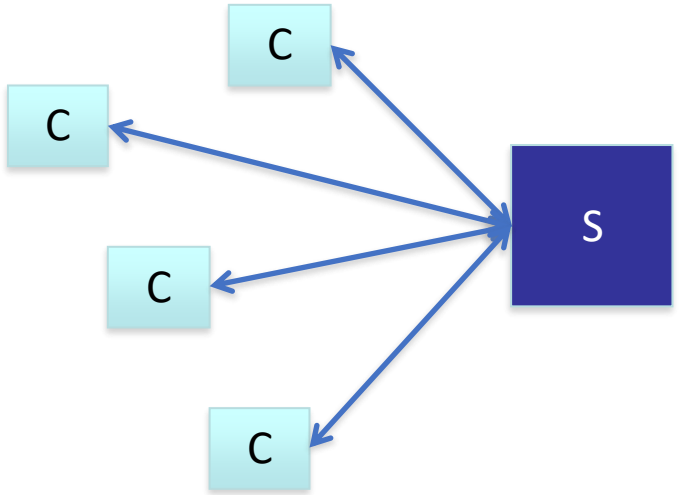
Web at 100,000 feet

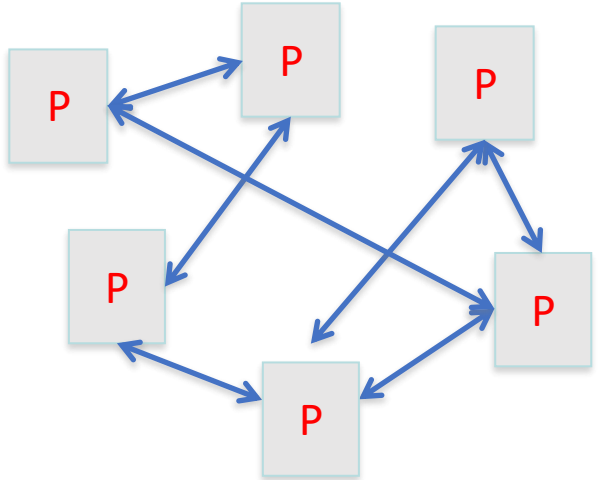
- The web is a *client/server* architecture
- It is fundamentally *request/reply oriented*
- Domain Name System (DNS) is another kind of server that maps *names* to *IP addresses*

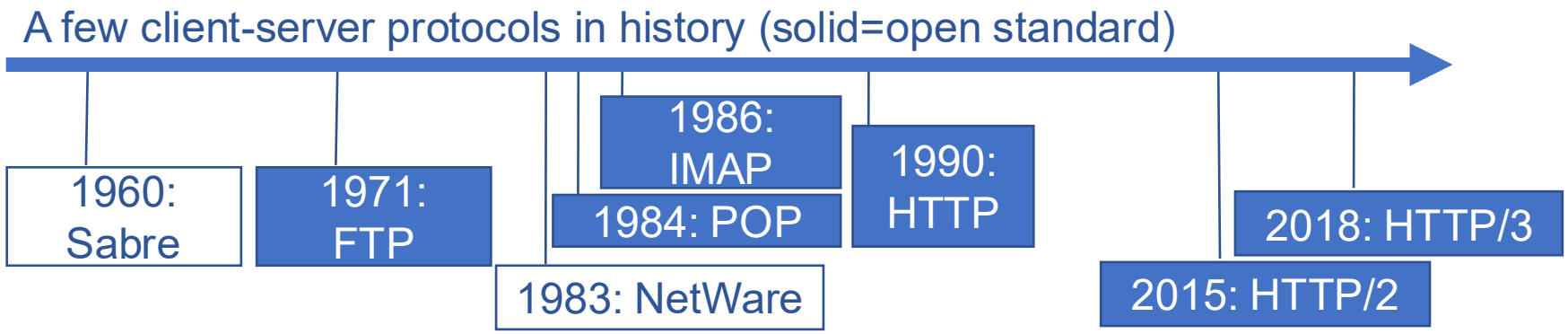


Client-Server is an Architectural *Design Pattern*

- Client & server each *specialized* for their tasks
  - Client: make queries on behalf of users
  - Server: await & respond to queries, serve many clients
  - Client-Server protocols are typically *request-reply*
  - *Frameworks* support client-side (React, iOS, etc.) and server-side (Rails, Django, etc.) app development
  - Clients used to be "simpler" than servers—now they're just differently complex!
- Peer-to-Peer: each participant is both a client and a server





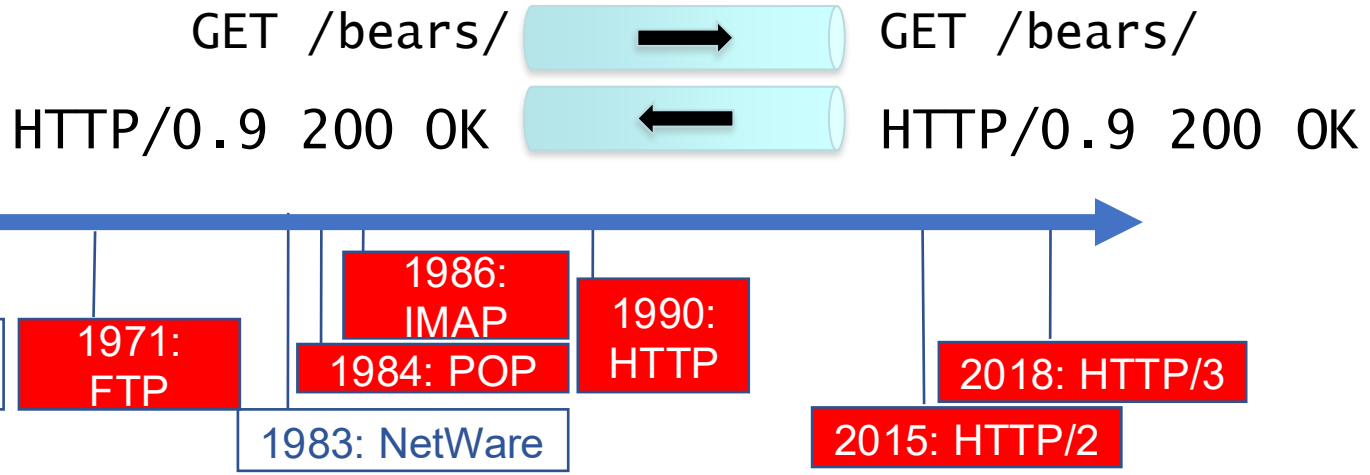


TCP/IP & HTTP (part 1/2: routes)

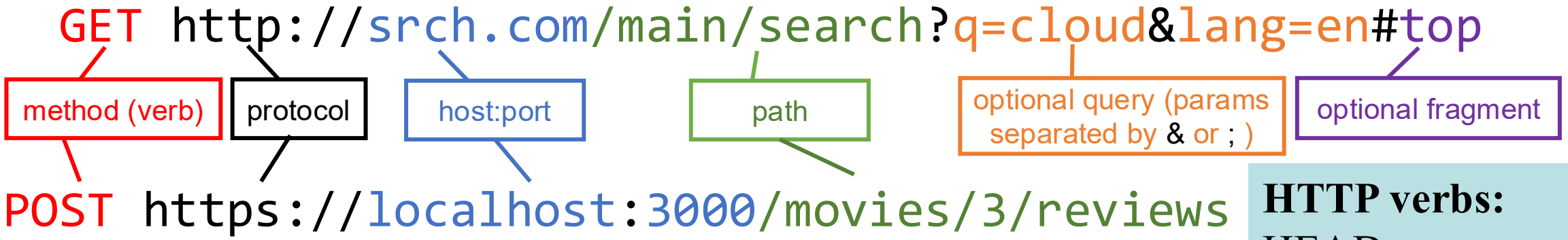
ESaaS §3.2

Nuts and bolts: TCP/IP protocols

- An IPv4 (Internet Protocol version 4) *address* identifies a physical network interface with four *octets,* e.g. `128.32.244.172`
  - Special address `127.0.0.1` is "this computer", named `localhost`, even if not connected to the Internet!
- TCP/IP (Transmission Control Protocol/Internet Protocol)
  - IP: no-guarantee, best-effort service that delivers *packets* from one IP address to another
  - TCP: make IP reliable by detecting "dropped" packets, data arriving out of order, transmission errors, slow networks, etc., and respond appropriately
  - TCP *ports* allow multiple TCP apps on same computer



Route == HTTP method + URI



- http? or https?
- Demo: a real HTTP request
  - also includes various *headers*
- Demo: a real HTTP reply

HTTP verbs:
HEAD—*get metadata*
GET—*get data*
POST— *send data*
DELETE

HTTP status codes:
2xx — *all is well*
3xx — *resource moved*
4xx — *access problem*
5xx — *server error*

TCP/IP & HTTP (part 2/2: Cookies)

ESaaS §3.2

HTTP is a stateless protocol

- Every HTTP request to the same server is independent of all prior requests!
- On first visit, server can include a `set-cookie` header in reply
  - Client's responsibility is to store the value and include in future request headers
  - "Incognito mode" destroys stored cookies when you quit browser

Cookie demo…

Fun fact: why is it called a cookie?

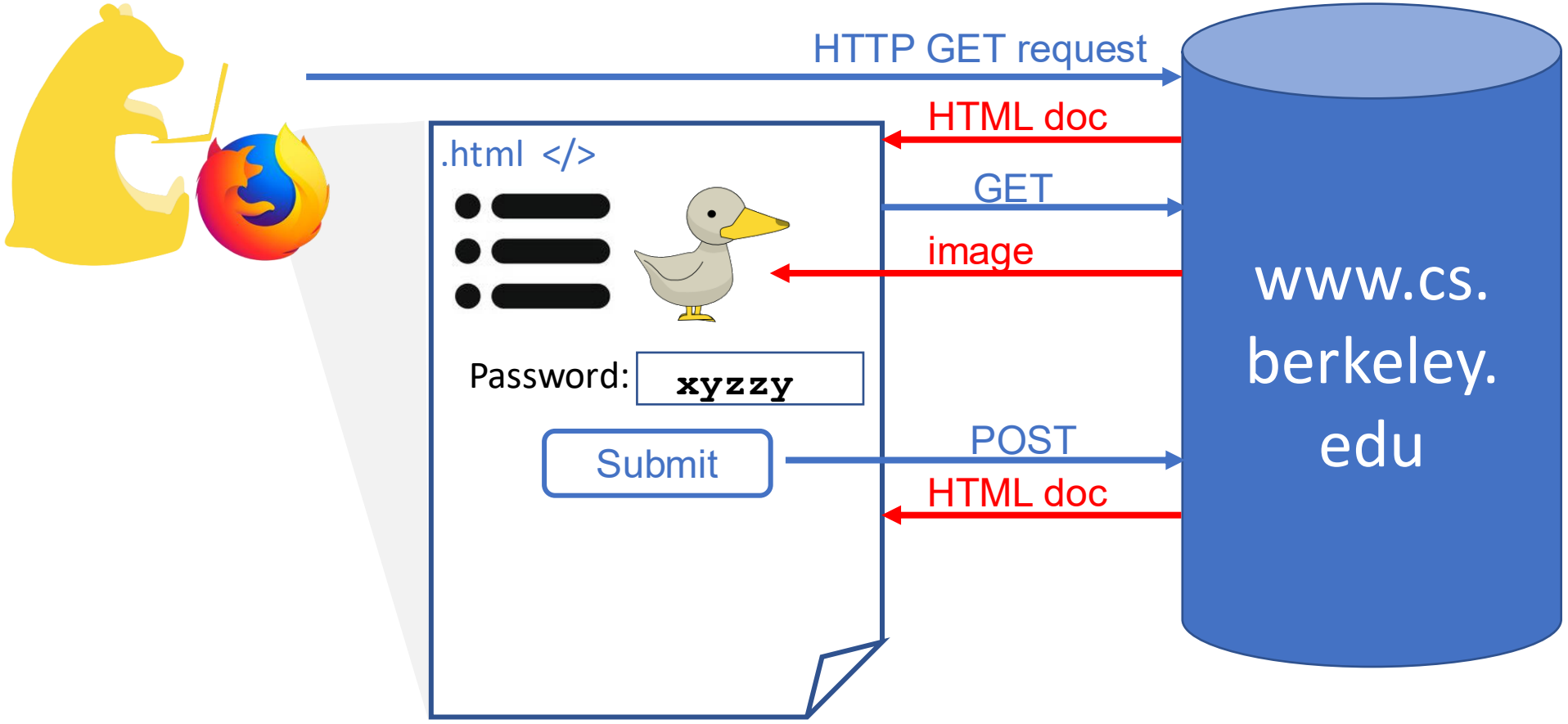
HTTP is a stateless protocol

- Name–value pair (string data)
- Session ID
- User ID
- User preferences (e.g., theme, language)
- Authentication token
- Tracking / analytics ID
- Expiration (Expires / Max-Age)
- Domain
- Path
- Secure, HttpOnly, SameSite attributes

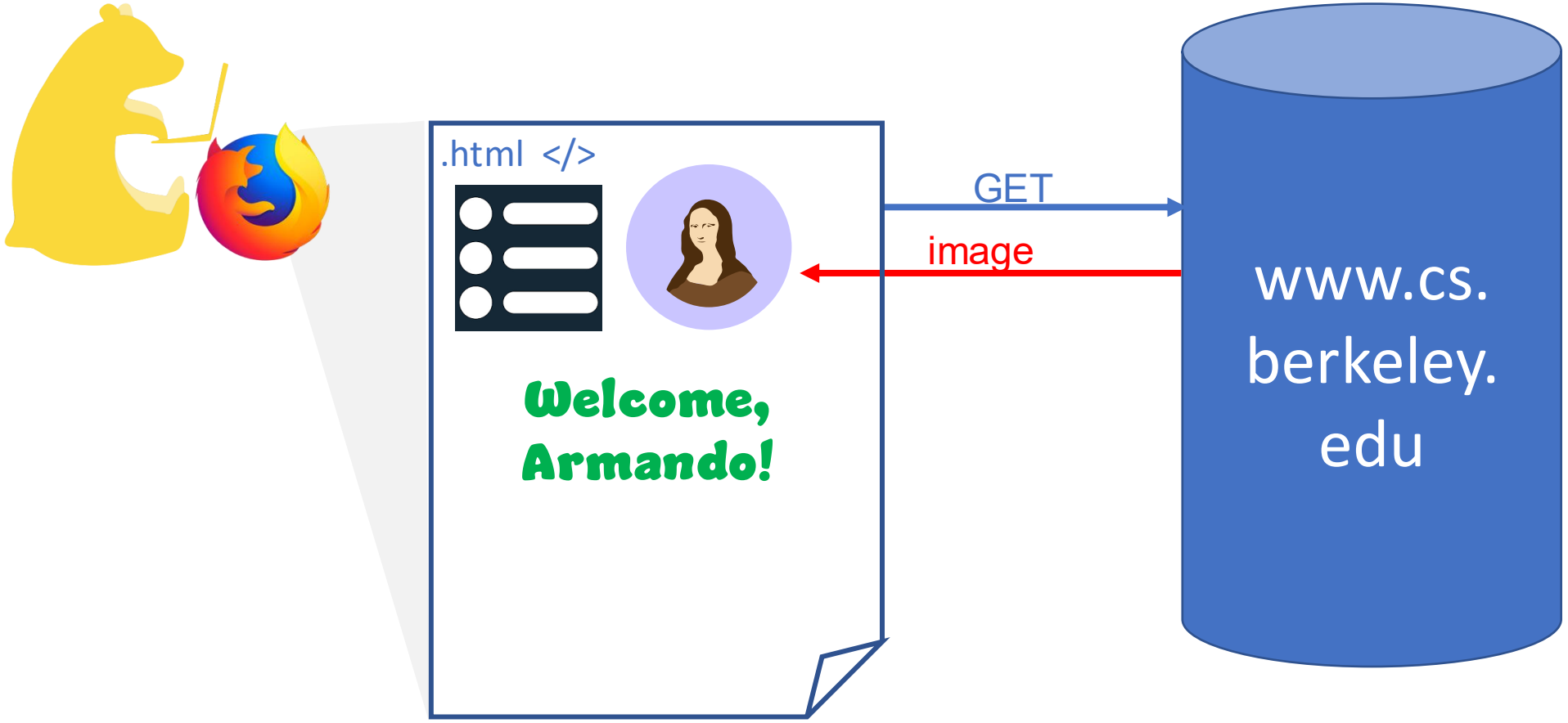
From Web Sites to Microservices: Service-Oriented Architecture

ESaaS §3.4

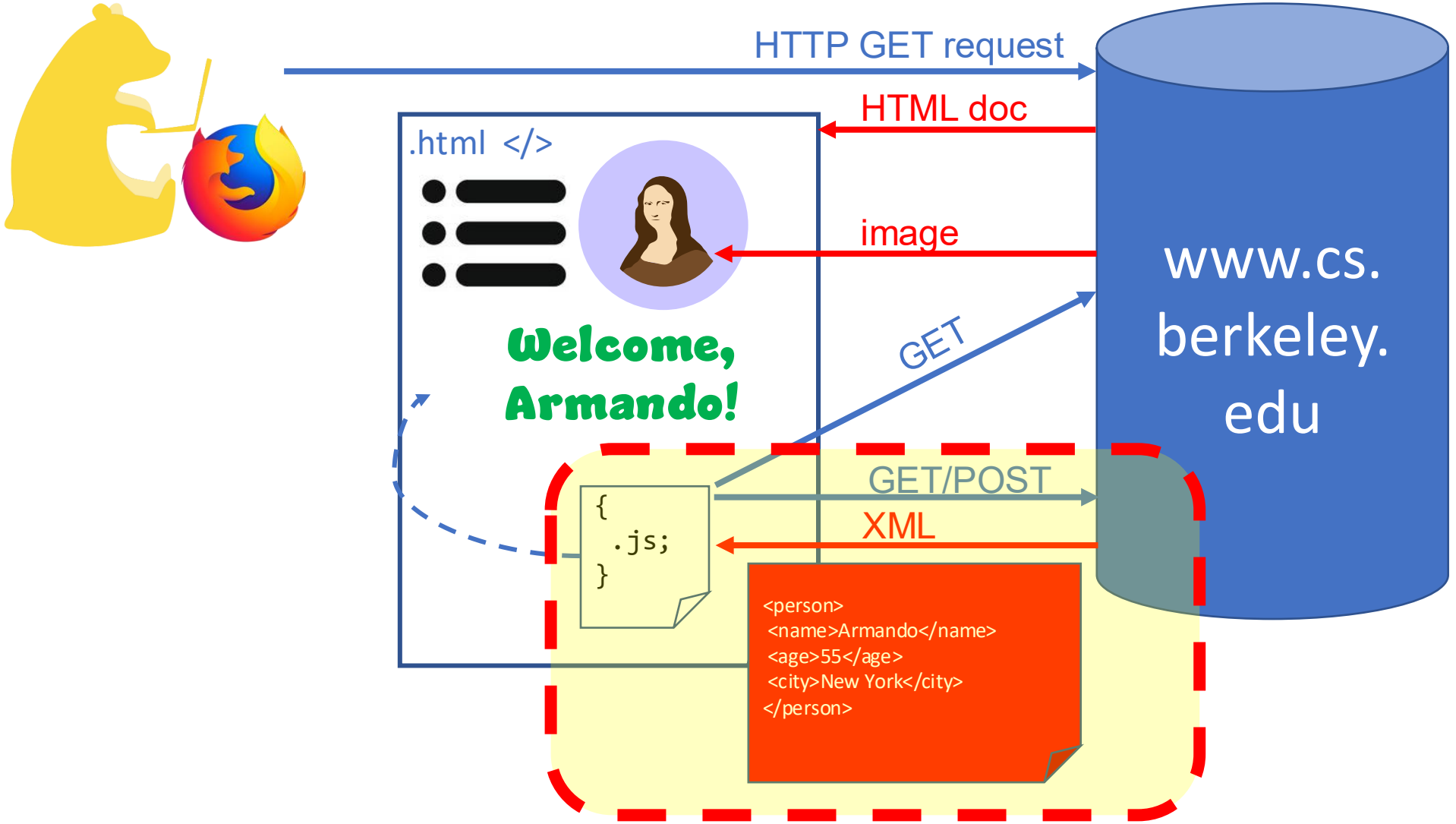
From Web 1.0 to 2.0



From Web 1.0 to 2.0



From Web 1.0 to 2.0



AJAX (Async. JavaScript and XML)

- Browser provides JavaScript language bindings for `XmlHttpRequest` and reply dispatch
  - nothing is XML-specific!
- *Microservice:* independently-deployable component that supports message-based communication
  - often not designed to be called from “top level”
  - may just provide “part of a page” (requires AJAX)

➔ highly specialized for one narrow task

Microservices  Agile?

- ✓ Small-to-medium projects work best
- ✓ “You build, you run”—developers close to customer, quick enhancements
- ✓ Compose small subsystems into a larger service

BUT:

- ! Performance
- ! Managing partial failure
- ! More interfaces to keep track of (but REST helps)
- ! Developers learn about operations, and vice versa

RESTful APIs: Everything Is a Resource

ESaaS §3.5

Procedure call vs. Microservice API call

• How to identify callee?	*endpoint*: base URI + path
• What operation?	path portion of URI
• How are arguments passed?	part of URI, or JSON/XML payload
• How is return value received?	JSON or XML data structure
• How are errors indicated?	HTTP status codes; returned data structure includes error message field

JavaScript Object Notation

- Primitive types: string, numeric, [array], {hash} (“object”)
- Usually, API response or payload is a single toplevel JSON object with well defined slots

```
<person>
 <name>John Doe</name>
 <age>30</age>
 <city>New York</city>
</person>
```

```
{
 "person": {
  "name": "John Doe",
  "age": 30,
  "city": "New York"
 }
}
```

REST: a canonical way of mapping URIs for remote procedure calls (Roy Fielding, 2000)

Everything the server manages is a resource.

- What resource is affected by this API call?
- What is done to the resource? What are possible results & side effects?
- What other data is needed?

Canonically: Create, Read, Update, Delete, Index (list)

always use POST or PUT

RESTful URIs, API Calls, and JSON TMDb Example 1 of 2

ESaaS §3.6

Example: TheMovieDB API

- Docs: https://developer.themoviedb.org/reference
- API endpoint: `https://api.themoviedb.org/3`
- Example call: `GET /movie/{movie_id}`
- Authentication: in HTTP Authorization header

```
Authorization: Bearer {api-key}
```

```
curl --request GET \
     --url 'https://api.themoviedb.org/3/movie/62' \
     --header 'Authorization: Bearer eyJhb...0kjM7o' \
     --header 'accept: application/json'
```

RESTful URIs, API Calls, and JSON TMDb Example 2 of 2

ESaaS §3.6

Another example: post “rating”

```
POST /movie/{movie_id}/rating
```

- Why POST?

```
curl --request POST \
     --url 'https://api.themoviedb.org/3/movie/62/rating' \
     --header 'Authorization: Bearer eyJhb...0kjM7o' \
     --header 'accept: application/json'
```

- How is actual rating info supplied?

Fallacies, Pitfalls, and Concluding Remarks

ESaaS §3.8–3.9

Reading API docs is a life skill

- How is authentication done?
- Use `curl` to try things out
- Use `jq` or similar tool to inspect JSON output
- …even if the API has native *language bindings*

“Resource-first” vs “action-first”

1. search for product
2. add product to cart
3. add payment/ shipping
4. finalize purchase

Create Cart resource

Get index of products, possibly filtered

Update Cart resource with product ID

Create Fulfillment resource with payment/shipping

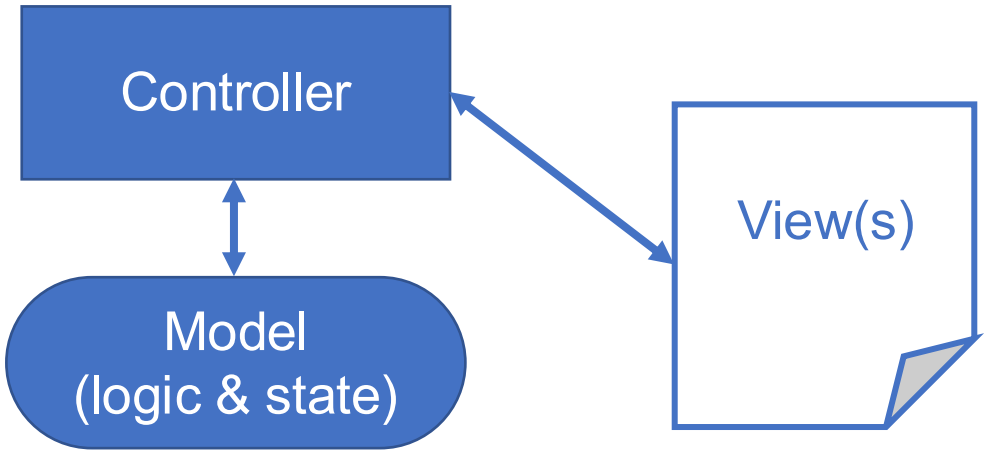
Create Order resource linked to Cart and Fulfillment

The Model-View-Controller (MVC) Architectural Design Pattern

ESaaS §4.1

Model-View-Controller: a design pattern

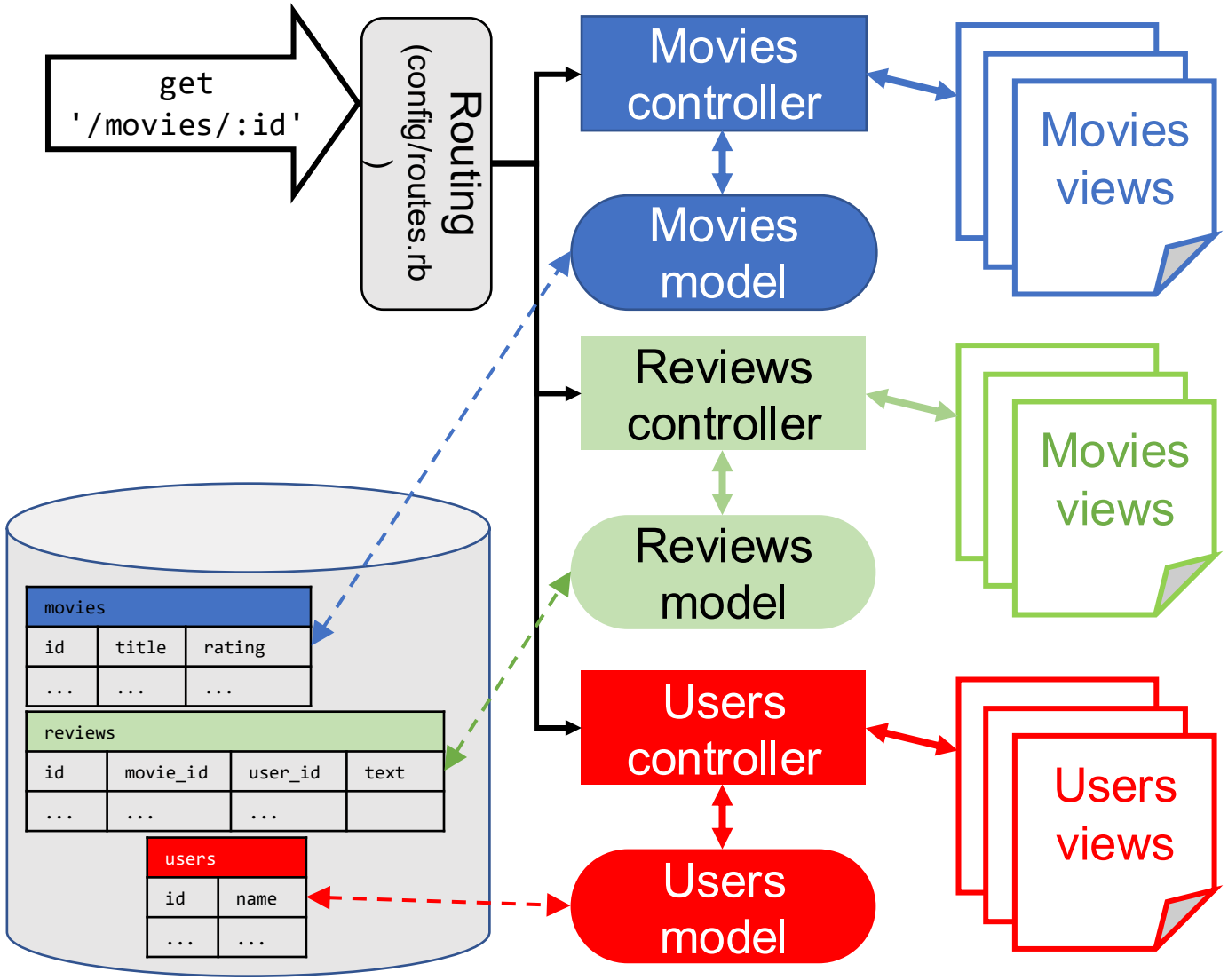
- Model: where the action is
  - state: what is stored about the model
  - logic: operations on model state
- View: lets user see and interact with the model
- Controller: mediates those interactions
  - updates view when model state changes
  - invokes model logic in response to user actions
- What does the design pattern *not* specify?
  - how or where the model state is stored
  - the technology or nature of the “connections” among M, V, and C



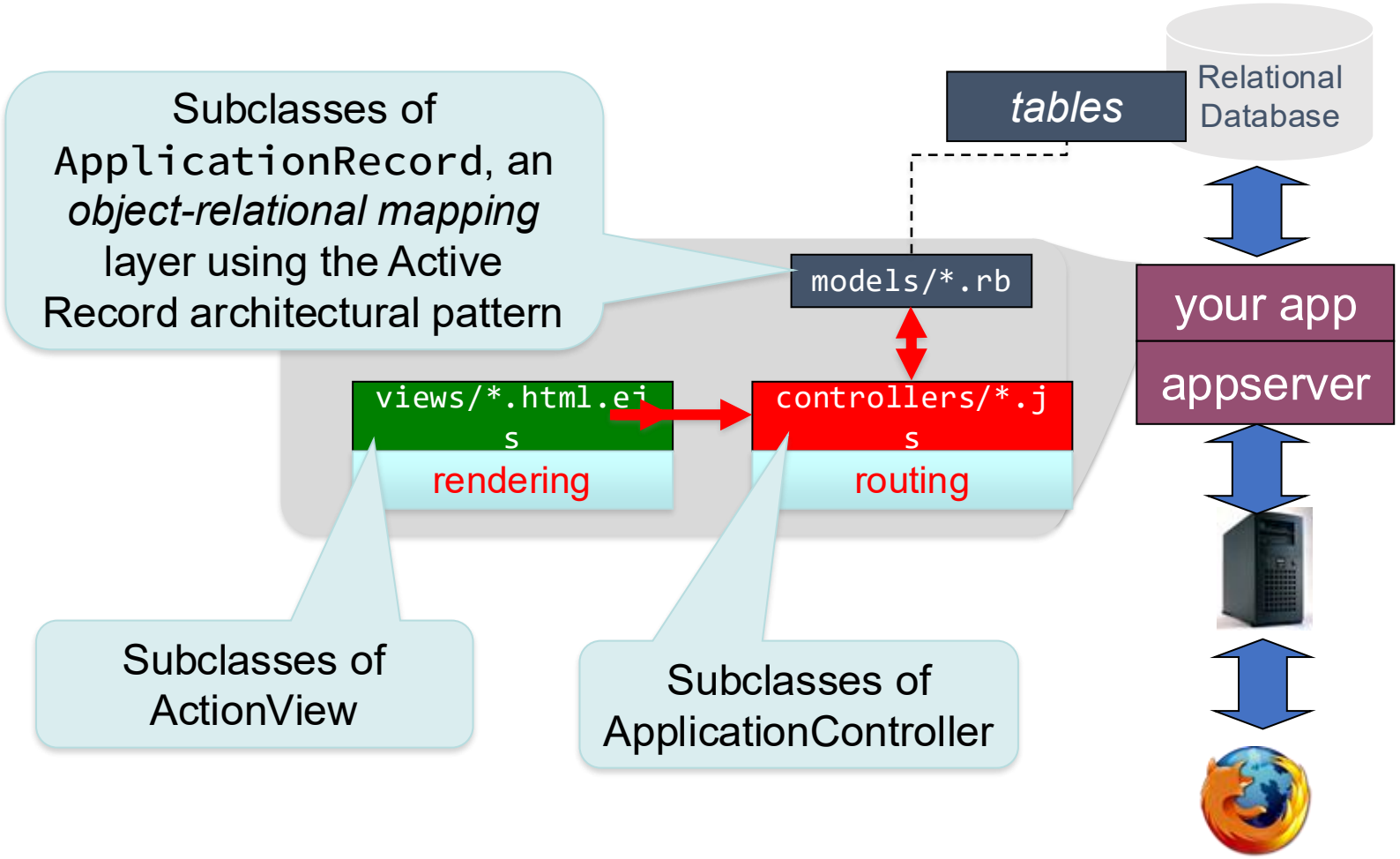
Express.js as an MVC Framework

Express: an MVC-based framework

- App server passes request route to Rails
- *Routing subsystem* parses params, identifies matching action
- Controller action is called
- Model state stored/updated in relational database tables
- View is rendered
- This is the basic scheme—variations exist
- ***Everything here is stateless except the database!***



Express as an MVC Framework



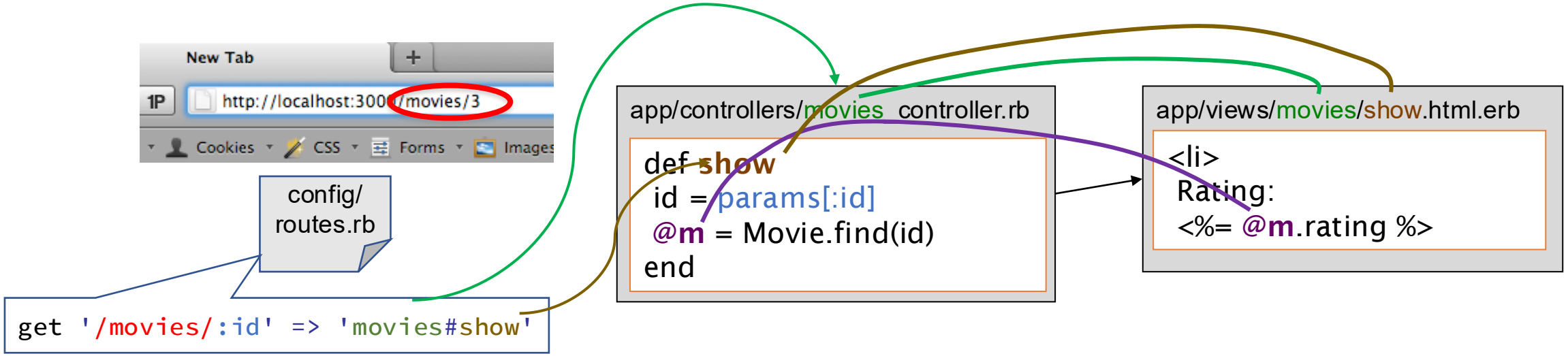
Develop	Deploy
Sqlite3	PostgreSQL
Node.js	puma, unicorn
Node.js	cowboy
your computer or Codio IDE	Heroku's cloud

Putting it all together: A trip through an Express app

ESaaS §4.1

A Trip Through An Express App

1. Routes map incoming URL's to *controller actions* and extract any optional *parameters*
2. Controller actions consume parameters and set *instance variables*, visible to *views*
3. Controller action eventually *renders* a view
   - Subdirs and filenames of **`views/`** match controllers & action names



Express Philosophy: C/C & DRY

- Convention over configuration
  - If naming follows certain conventions, no need for config files

`MoviesController#show` in **`movies_controller.js`**
`views/movies/show.html.ejs`

- Don't Repeat Yourself (DRY)
  - Mechanisms to extract common functionality
  - e.g.: route info in a single file is also used to construct forms that submit to an action!

Models As Resources

ESaaS§4.2

Resource ⇔ Model

- What can you do to resource?
  - CRUD
- How to name resource & operation?
  - route
- How to inspect/modify resource?
  - views & controllers
- Where does the resource's state live?
  - model persisted in database table

A model is backed by a database

```
class Account
  def initialize(starting_balance=0)
    @balance = starting_balance
  end
  attr_reader :balance
  def withdraw(amount) ... end
  def deposit(amount) ... end
end
```

table “accounts”

column

model logic

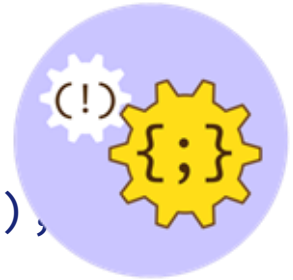
```
account1 = Account.new(1000)
account2 = Account.new(500)

account1.withdraw(200)
puts "Balance: #{account1.balance}"
```

row

- Collection of instances ⇔ database table
- 1 instance ⇔ 1 row (model state)
- Attribute of model ⇔ database column
- Class & instance methods ⇔ model logic

CRUD in SQL

- Structured Query Language (SQL) is the query language used by RDBMS's
- 4 basic operations on a table row:
  Create, Read, Update attributes, Delete




```
CREATE TABLE movies (id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255),
                     rating VARCHAR(255)

INSERT INTO movies (title, rating)
     VALUES ("The Thing", "R"),
            ("Coco", "PG"),
            ("Lost In Translation", "R")

SELECT *
    FROM movies
    WHERE (rating = "R")

UPDATE movies
    SET rating = "PG"
    WHERE title = "Coco"

DELETE FROM movies WHERE id=1
```

Models Store Data in Relational Databases (DBMS)

- Each type of model gets its own database *table*
  - All rows in table have identical structure
  - One row in table == one instance of model's class
  - Each column stores value of an *attribute* of the model instance
  - Each row has unique value for *primary key* (by convention, in Rails this is an integer and is called *id*)

id	rating	title	release_date
2	G	Gone With the Wind	1939-12-15
11	PG	Casablanca	1942-11-26
...	...	...	...
35	PG	Star Wars	1977-05-25

- *Database Schema:* Collection of all tables and their structure

MVC Responsibilities

- *Model:* methods to get/manipulate data

```
Movie.where(...), Movie.find(...)
```

Instance variables set in Controller available in View

- *Controller:* get data from Model, make available to View

```
def show
  @movie = Movie.find(params[:id])
end
```

By default, Rails will look for app/views/movies/show.html.erb

- *View:* display data, allow user interaction
  - Show details of a movie (description, rating)
- But…
  - What else can user do from this page?
  - How does user get to this page?